Guia de Atividade Prática do Estudante: Design de Jogos de ED II

Disciplina: Estruturas de Dados II - Ciência da Computação

Tema: Design de Jogos Educativos sobre Árvores Avançadas por meio da Engenharia Reversa e Reuso de Modelos

1. Identificação do Grupo

Integrantes: Breno Silva Marinho

Data: 18/09/2026

Turma: Ciência da Computação

2. Orientações Gerais e Objetivos

Nesta atividade prática baseada em projetos, seu grupo assumirá o papel de uma equipe de Game Designers especializados em softwares educativos. O objetivo principal é analisar um jogo ou visualizador de estruturas de dados existente na literatura ou na web, identificar suas limitações pedagógicas/técnicas e projetar um upgrade de mecânica incorporando estruturas de dados avançadas (Árvores AVL, Árvores Rubro-Negras ou Árvores B/B+).

Dica de Pesquisa e Reuso: Busque na ciência e na comunidade o que já foi feito (ex.: VisuAlgo, jogos estilo Tower Defense para BST, puzzles de ordenação) e reuse o modelo existente de forma a propor uma melhoria funcional ou pedagógica ativa.

3. Estrutura e Roteiro do Workshop (80 min)

- **Fase 1 - Mapeamento e Engenharia Reversa (15 min):** Escolha do jogo/ferramenta base e análise de limitações.
- **Fase 2 - Elaboração do Game Design Document Briefing (35 min):** Preenchimento do template de especificação e tradução dos conceitos de ED II para o gameplay.
- **Fase 3 - Preparação e Pitch de Apresentação (30 min):** Apresentação de um Pitch de 3 minutos para a turma com defesa do modelo.

4. Template de Especificação: Game Design Document Briefing

4.1 Diagnóstico do Modelo Reutilizado

Nome do Jogo/Ferramenta Base Identificado: Sortia 2.0 - Um jogo de ordenação para ensino de Estrutura de Dados (HeapSort)

Limitação Didática ou Técnica Identificada no Modelo Original:

O Sortia foi desenvolvido especificamente para ensinar o algoritmo HeapSort por meio da simulação de sua execução, sendo um jogo on-line, gratuito e single player. Nosso grupo, entretanto, não conseguiu acesso ao código original do Sortia, motivo pelo qual, foi desenvolvido outra versão para jogo diretamente no console.

O Sortia 2.0 apresenta uma abordagem eficiente para a aprendizagem praticada do HeapSort, permitindo ao jogador manipular os ponteiros "Pai" e "Filho" e executar as operações necessárias à ordenação. Entretanto, sua mecânica está restrita à representação de um vetor interpretado como heap e à execução de operações especificas do heapsort.

Entretanto, foi observada uma limitação para o jogador durante a execução da partida, visto que, a estrutura hierárquica da árvore não é apresentada em momento algum, sendo restrita a uma apresentação em forma de vetor, o que dificulta a compreensão inicial da estrutura pretendida. Assim, o jogador pratica a comparação e as trocas entre pai e filho mas não desenvolve competências acerca da estrutura e manutenção dinâmica do balanceamento da árvore.

Uma possível atualização propõe preservar a aprendizagem ativa do Sortia, mas transformar o jogador em responsável pela construção e manutenção de uma árvore AVL balanceada.

4.2 Seleção da Estrutura de ED II para o Upgrade

Selecione a estrutura avançada que fundamentará a nova mecânica do jogo:

[X] Árvore AVL:	Focado no Fator de Balanceamento FB em {−1, 0, +1} e execução de Rotações Simples/Duplas.
[ ] Árvore Rubro-Negra:	Focado nas propriedades das cores dos nós, altura preta e recoloramento/rotação.
[ ] Árvore B/B+:	Focado no grau mínimo m, limite de chaves por página/nó e operações de Split (divisão) e Merge (fusão).

4.3 Mapeamento de Conceitos em Mecânicas de Gameplay

Conceito Teórico de ED II	Elemento / Mecânica Correspondente no Jogo
Nó da Árvore / Chave	Cada número sorteado pelo jogo representa uma chave armazenada em um nó da Árvore AVL. O jogador deve inserir a chave respeitando a propriedade da árvore binária de busca: valores menores seguem para a subárvore esquerda e valores maiores para a direita.
Altura da Árvore (h)	Cada nó apresenta sua altura atual. Após uma inserção, o jogador deve observar/recalcular as alturas no caminho entre o novo nó e a raiz. A altura influencia diretamente o cálculo do fator de balanceamento.
Fator de Balanceamento	Cada nó possui um indicador de equilíbrio, calculado pela diferença entre as alturas de suas subárvores. Valores permitidos em uma AVL balanceada são −1, 0 e +1. Quando o módulo do FB ultrapassa 1, o indicador sinaliza que aquele nó necessita de correção.
Operação de Correção	O jogador deve identificar o tipo de desbalanceamento e selecionar a operação adequada: LL → rotação simples à direita; RR → rotação simples à esquerda; LR → rotação dupla esquerda-direita; RL → rotação dupla direita-esquerda. Após a rotação correta, a árvore é reorganizada e as alturas e fatores de balanceamento são atualizados.

4.4 Regras do Core Loop e Condições do Jogo

Core Loop: A cada rodada, o jogador precisa identificar o tipo de desbalanceamento ocorrido e realizar a troca de valores de pai e filho continuamente até que o valor correto esteja na posição ideal, seguindo as regras da Árvore Binária de Busca. Após a inserção correta, o jogador deverá percorrer o caminho de novo do nó até a raiz, verificando as alturas das subárvores, organizando novamente a estrutura.

Após a correção, o jogo atualiza a estrutura e o jogador prossegue para a próxima chave. Acertos consecutivos geram pontos e bônus, enquanto inserções ou rotações incorretas geram feedback pedagógico e penalização.

Condição de Vitória: O jogador vence ao inserir corretamente todas as chaves propostas e concluir a partida mantendo a Árvore AVL válida e balanceada, isto é, preservando simultaneamente a propriedade de Árvore Binária de Busca e garantindo que todos os nós apresentem fator de balanceamento pertencente a {−1, 0, +1}. A pontuação final considera o número de inserções corretas, identificação dos desbalanceamentos, escolha das rotações adequadas, quantidade de erros e tempo de execução.

Condição de Derrota: O jogador perde a partida ao atingir o limite de erros permitido e deixar de corrigir sucessivos desbalanceamentos da árvore. São considerados erros críticos: inserir uma chave violando a propriedade da Árvore Binária de Busca; aceitar um nó com $|FB| > 1$ sem executar a correção; ou aplicar uma rotação incompatível com o caso de desbalanceamento identificado. A derrota representa pedagogicamente a degradação estrutural da árvore, pois a ausência de balanceamento pode aumentar excessivamente sua altura e comprometer a eficiência das operações de busca, inserção e remoção, aproximando seu comportamento do pior caso de uma árvore de busca não balanceada.

5. Checklist de Autoavaliação do Grupo

[X] Identificamos e citamos o jogo/modelo base existente na literatura ou na web.

[X] O upgrade proposto exige a aplicação prática de conceitos de ED II (AVL, Rubro-Negra ou Árvore B).

[X] As propriedades algorítmicas (ex.: Fator de Balanceamento) estão diretamente traduzidas em mecânicas ativas de jogo.

[X] A condição de derrota está associada ao pior caso de complexidade ou quebra das regras da estrutura.

[X] O grupo está preparado para realizar a defesa do projeto em um Pitch de 3 minutos.